

DRIVE EVENT

Para ilustrar el uso conjunto y correcto de los paquetes DBMS_AQADM, DBMS_AQELM y UTL_HTTP, plantearemos el siguiente Caso de Uso del Mundo Real.

El Escenario: Sistema de Webhooks para Pasarela de Pagos (Event-Driven)

Una base de datos de comercio electrónico procesa pagos. Cada vez que un pago se aprueba, se genera un evento de negocio de manera asíncrona. La base de datos debe:

1. DBMS_AQADM: Administrar la infraestructura de colas seguras donde residen estos mensajes de pago.
2. DBMS_AQELM: Configurar el entorno de red global de mensajería (como un servidor Proxy empresarial) para que las notificaciones salientes viajen adecuadamente.
3. UTL_HTTP: Actuar como el motor interno del procedimiento de suscripción (PL/SQL Callback). Cuando un mensaje entra a la cola, el callback se dispara automáticamente, extrae el payload y realiza una llamada HTTP POST segura hacia la API externa del cliente (Webhook) para notificarle "Pago Procesado".

DESARROLLO

1. SetUP

Configuración de usuario

```
--  
-- "/ as sysdba  
--  
CREATE USER aq IDENTIFIED BY aq;  
  
GRANT CONNECT, RESOURCE, aq_administrator_role TO aq;  
  
GRANT EXECUTE ON dbms_aq TO aq;  
  
BEGIN  
    --  
    dbms_aqadm.grant_system_privilege('ENQUEUE_ANY', 'AQ', FALSE);  
    dbms_aqadm.grant_system_privilege('DEQUEUE_ANY', 'AQ', FALSE);  
    --  
END;
```

2. Configurar la Infraestructura de Mensajería (DBMS_AQADM)

El administrador de colas utiliza DBMS_AQADM para crear la tabla de colas (Queue Table) y la cola física (Queue), activando además la posibilidad de múltiples consumidores si fuera necesario.

```
-- Ejecutado por un usuario con rol de administración de AQ
BEGIN
  /*
    APP_USER: es el esquema del usuario
  */
  --
  -- 1. Crear la Tabla de la Cola (Donde se almacenan físicamente los
mensajes)
  DBMS_AQADM.CREATE_QUEUE_TABLE(
    queue_table      => 'APP_USER.pagos_queue_tab',
    queue_payload_type => 'RAW', -- Usamos RAW para manejar texto plano,
JSON o XML directamente
    multiple_consumers => TRUE   -- Permite que múltiples sistemas se
suscriban al evento
  );
  --
  -- 2. Crear la Cola propiamente dicha
  DBMS_AQADM.CREATE_QUEUE(
    queue_name  => 'APP_USER.pago_procesado_queue',
    queue_table => 'APP_USER.pagos_queue_tab'
  );
  --
  -- 3. Iniciar la Cola para permitir encolar y desencolar mensajes
  DBMS_AQADM.START_QUEUE(
    queue_name => 'APP_USER.pago_procesado_queue'
  );
END;
```

3. Parametrizar el Entorno de Transporte de Mensajería (DBMS_AQELM)

Antes de que las transacciones y notificaciones comiencen a fluir, se debe garantizar el enrutamiento de la red interna de base de datos hacia internet mediante DBMS_AQELM. Esto configura la infraestructura subyacente para los subsistemas de mensajería asíncrona.

```
BEGIN
  --
  -- Si el servidor de Base de Datos está detrás de un proxy corporativo para
salir a internet,
  -- configuramos el proxy global y evitamos que use el proxy para dominios
locales.
  DBMS_AQELM.SET_PROXY(
    proxy           => '://miempresa.com',
    no_proxy_domains => '*.miempresa.local, intranet'
  );
  --
END;
```

4. Crear el Consumidor Automático (Callback con UTL_HTTP)

Aquí es donde aplicamos UTL_HTTP. En lugar de usar una notificación HTTP directa nativa de AQ (que carece de la opción de estructurar headers complejos), creamos un procedimiento almacenado PL/SQL Callback. Este procedimiento será invocado asíncronamente por Oracle AQ cada vez que entre un mensaje. El procedimiento consume el mensaje de la cola y usa UTL_HTTP para hacer un POST HTTP estructurado enviando el JSON del pago.

```
/*
  APP_USER: es el esquema del usuario.
  Los parametros de este procedimiento son especificos de la firma de
*/
CREATE OR REPLACE PROCEDURE app_user.webhook_pago_callback(
    context IN RAW,
    reginfo IN SYS.AQ$_REG_INFO,      -- Corregido: Lleva guion bajo
    descr   IN SYS.AQ$_DESCRIPTOR,
    payload IN RAW,
    payload1 IN NUMBER                -- Corregido: Es un NUMBER (Longitud
del payload)
) IS
--
    r_enqueue_options DBMS_AQ.ENQUEUE_OPTIONS_T;
    r_dequeue_options DBMS_AQ.DEQUEUE_OPTIONS_T;
    r_message_properties DBMS_AQ.MESSAGE_PROPERTIES_T;
    v_message_handle    RAW(16);
    v_payload           RAW(32767);
--
    -- Variables para UTL_HTTP
    v_http_req          UTL_HTTP.REQ;
    v_http_resp         UTL_HTTP.RESP;
    v_url               VARCHAR2(1000) := 'https://clienteexterno.com';
    v_json_data         VARCHAR2(32767);
--
BEGIN
--
    -- [El resto de la lógica del cuerpo del procedimiento se mantiene
idéntica]
    r_dequeue_options.msgid := descr.msg_id;
    r_dequeue_options.consumer_name := descr.consumer_name;
--
    DBMS_AQ.DEQUEUE(
        queue_name      => descr.queue_name,
        dequeue_options => r_dequeue_options,
        message_properties => r_message_properties,
        payload         => v_payload,
        msgid           => v_message_handle
    );
--
    v_json_data := UTL_RAW.CAST_TO_VARCHAR2(v_payload);
--
```

```

    UTL_HTTP.SET_PROXY('://miempresa.com', NULL);
    --
    v_http_req := UTL_HTTP.BEGIN_REQUEST(v_url, 'POST', 'HTTP/1.1');
    --
    UTL_HTTP.SET_HEADER(v_http_req, 'Content-Type', 'application/json');
    UTL_HTTP.SET_HEADER(v_http_req, 'Authorization', 'Bearer
TOKEN_SECRETO_WEBHOOK');
    UTL_HTTP.SET_HEADER(v_http_req, 'Content-Length',
TO_CHAR(LENGTH(v_json_data)));
    UTL_HTTP.WRITE_TEXT(v_http_req, v_json_data);
    v_http_resp := UTL_HTTP.GET_RESPONSE(v_http_req);
    --
    UTL_HTTP.END_RESPONSE(v_http_resp);
    --
EXCEPTION
    WHEN OTHERS THEN
        UTL_HTTP.END_RESPONSE(v_http_resp);
        RAISE;
END webhook_pago_callback;

```

5. Unir las piezas (Suscripción y Registro del Callback)

Volvemos a utilizar DBMS_AQADM para añadir un suscriptor a nuestra cola multi-consumidor. Luego, usando el paquete de ejecución estándar DBMS_AQ, registramos el procedimiento almacenado (creado en el Paso 3 con UTL_HTTP) para que escuche activamente esa suscripción.

```

DECLARE
    --
    -- Son objetos de Oracle
    v_subscriber      SYS.AQ$_AGENT;
    v_subscription    SYS.AQ$_REG_INFO;
    v_sub_list        SYS.AQ$_REG_INFO_LIST;
    --
BEGIN
    --
    -- 1. Definir (Instanciar) y añadir el suscriptor a la cola vía DBMS_AQADM
    v_subscriber := SYS.AQ$_AGENT('suscriptor_webhooks', NULL, NULL);
    --
    DBMS_AQADM.ADD_SUBSCRIBER(
        queue_name => 'APP_USER.pago_procesado_queue',
        subscriber => v_subscriber
    );
    --
    -- 2. Vincular el suscriptor con el procedimiento PL/SQL Callback
    -- El formato de la suscripción debe ser: SCHEMA.QUEUE:SUBSCRIBER_NAME
    v_subscription := SYS.AQ$_REG_INFO(
        name      => 'APP_USER.pago_procesado_queue:suscriptor_webhooks',
        namespace => DBMS_AQ.NAMESPACE_AQ, -- Namespace estándar de colas
        callback  => 'plsql://APP_USER.webhook_pago_callback', -- Apunta al
procedimiento con UTL_HTTP
        context   => RAWTOHEX('INFO_CONTEXTO_OPCIONAL')
    );

```

```

);
--
-- 3. Registrar la suscripción en el motor de Base de Datos
v_sub_list := SYS.AQ$_REG_INFO_LIST(v_subscription);
--
DBMS_AQ.REGISTER(
    reg_list => v_sub_list,
    reg_count => 1
);
--
END;

```

Resumen del Flujo de Datos

1. Tu aplicación inserta un mensaje en APP_USER.pago_procesado_queue usando DBMS_AQ.ENQUEUE y hace COMMIT.
2. El monitor de AQ detecta el mensaje e identifica que suscriptor_webhooks (creado con DBMS_AQADM) está interesado.
3. El motor levanta un proceso asíncronos respetando las propiedades de proxy definidas por DBMS_AQELM.
4. Se ejecuta el procedimiento webhook_pago_callback.
5. UTL_HTTP emite el JSON del mensaje hacia el endpoint HTTPS externo de forma segura y eficiente.
[1] (https://docs.oracle.com/en/database/oracle/oracle-database/18/arpls/DBMS_AQADM.html),

NOTA: Recuerda que si es HTTPS, necesitaremos configurar adicionalmente un Oracle Wallet para el paquete UTL_HTTP

CONFIGURACION Y SEGURIDAD ADICIONAL

A continuación, se presentan los scripts y pasos necesarios para configurar los permisos de red (ACLs) y el Oracle Wallet para habilitar de forma segura las peticiones HTTPS con el paquete UTL_HTTP. Los scripts de ACL utilizan la sintaxis recomendada para Oracle 12c, 19c, 21c y versiones superiores (APPEND_HOST_ACE)

Configurar las ACLs (Access Control Lists)

Desde Oracle 12c en adelante, ya no es necesario crear archivos XML manualmente. El administrador (SYSDBA) utiliza el procedimiento APPEND_HOST_ACE para otorgar permisos de conexión a hosts específicos.

Ejecuta este bloque como SYS o SYSTEM:

```

BEGIN
--
-- 1. Permiso para conectarse al Host del API externa (Puerto 443 para
HTTPS)

```

```

DBMS_NETWORK_ACL_ADMIN.APPEND_HOST_ACE(
  host          => '://clienteexterno.com', -- Cambia por tu dominio o
IP
  lower_port     => 443,
  upper_port     => 443,
  ace           => xs$ace_type(
                    privilege_list => xs$name_list('connect',
'resolve'),
                    principal_name => 'APP_USER',           -- Tu
usuario de BD
                    principal_type => xs_acl.ptype_db
                )
);
--
-- 2. Permiso para pasar a través del Proxy (Puerto 8080 del ejemplo
anterior)
DBMS_NETWORK_ACL_ADMIN.APPEND_HOST_ACE(
  host          => '://miempresa.com',      -- Cambia por tu servidor
Proxy
  lower_port     => 8080,
  upper_port     => 8080,
  ace           => xs$ace_type(
                    privilege_list => xs$name_list('connect',
'resolve'),
                    principal_name => 'APP_USER',
                    principal_type => xs_acl.ptype_db
                )
);
--
END;

```

Configurar el Oracle Wallet (Para HTTPS/SSL)

Para que UTL_HTTP pueda validar certificados SSL en conexiones HTTPS, requiere una firma digital de confianza. Tienes dos opciones dependiendo de la arquitectura y la versión exacta de tu base de datos:

Opción A: El método moderno sin archivos físicos (Recomendado para Oracle 19c en adelante)
 Las versiones modernas de Oracle 19c (actualizadas) incorporan un backport que permite utilizar directamente el almacén de certificados del sistema operativo (OS Certificate Store). No necesitas crear archivos en el servidor. Solo debes añadir esta línea al inicio de tu bloque o procedimiento PL/SQL antes del BEGIN_REQUEST [Referencia](#)

```

-- Le indica a Oracle que confíe en los certificados SSL globales del sistema
operativo
UTL_HTTP.SET_WALLET('system:', NULL);

```

Opción B: Crear un Oracle Wallet físico (Método Tradicional)

Si tu servidor requiere un Wallet manual debido a políticas estrictas o versiones específicas de TLS,

sigue estos pasos desde la consola del servidor de base de datos (con el usuario de sistema oracle): [rostantechnologies](#)

1. Descargar el certificado raíz del sitio destino Entra a la URL (<https://clienteexterno.com>) desde un navegador o usa openssl para guardar el certificado raíz e intermedio de la entidad emisora (CA) en formato .cer o .pem. Pásalos al servidor de base de datos. [rostantechnologies](#)
2. Crear el directorio y el Wallet vacío (vía terminal de Linux/Windows) [rostantechnologies](#)

```
# LINUX
# Crear la carpeta física
mkdir -p /opt/oracle/my_wallet

# Crear el wallet con la opción auto-login habilitada
orapki wallet create -wallet /opt/oracle/my_wallet -password
"MiPasswordSecreto123" -auto_login
```

3. Importar el certificado de confianza [rostantechnologies](#)

```
orapki wallet add -wallet /opt/oracle/my_wallet -password
"MiPasswordSecreto123" -trusted_cert -cert /ruta/certificado_raiz.crt
```

4. Otorgar accesos a nivel de base de datos Debes registrar el Wallet en la base de datos y darle permisos al usuario de la aplicación para utilizarlo: [forums Oracle](#)

```
-- Ejecutado como SYSDBA
BEGIN
--
    DBMS_NETWORK_ACL_ADMIN.APPEND_WALLET_ACE(
        wallet_path    => 'file:/opt/oracle/my_wallet',
        ace             => xs$ace_type(
            privilege_list =>
xs$name_list('use_client_certificates'),
            principal_name => 'APP_USER',
            principal_type => xs_acl.ptype_db
        )
    );
--
END;
```

5. ¿Cómo se aplica en el código PL/SQL?

Modificando ligeramente el procedimiento webhook_pago_callback visto anteriormente, así se invocaría el Wallet físico de manera correcta antes de abrir el canal HTTP: [rostantechnologies](#)

```
-- ... [Dentro de la sección BEGIN del procedimiento de Webhooks] ...
BEGIN
    -- 1. Apuntar al Oracle Wallet físico justo antes de iniciar la petición
    UTL_HTTP.SET_WALLET('file:/opt/oracle/my_wallet', NULL); -- NULL porque
    tiene auto_login

    -- 2. Configurar Proxy y continuar el flujo
    UTL_HTTP.SET_PROXY('://miempresa.com', NULL);
    v_http_req := UTL_HTTP.BEGIN_REQUEST(v_url, 'POST', 'HTTP/1.1');

    -- ... [El resto de cabeceras e inserciones de texto] ...
```

CONFIGURACION DE WINDOWS

Para aplicar esta configuración en un entorno de Windows Server, la lógica de las ACLs dentro de la base de datos se mantiene exactamente igual, pero cambian las rutas de archivos, la consola de comandos del sistema operativo y las variables de entorno.

A continuación, tienes la adaptación de la Parte 2 para crear e implementar el Oracle Wallet en Windows.

1. Crear la carpeta física para el Wallet Abre la consola de comandos de Windows (cmd) o PowerShell como Administrador en el servidor y ejecuta:

```
:: Crear el directorio en una ruta estándar de Windows
mkdir C:\app\oracle\product\wallets\my_wallet
```

2. Crear el Wallet usando orapki en Windows Asegúrate de que la variable de entorno %ORACLE_HOME% esté configurada en tu terminal de Windows. Ejecuta la herramienta integrada orapki:

```
:: Crear el wallet con la opción auto-login habilitada en Windows
orapki wallet create
    -wallet C:\app\oracle\product\wallets\my_wallet
    -password "MiPasswordSecreto123"
    -auto_login
```

3. Importar el certificado de confianza Una vez que hayas descargado el certificado raíz (por ejemplo, certificado_raiz.cer), impórtalo desde la consola de Windows:

```
:: Importar el certificado de la entidad certificadora (CA) externa
orapki wallet add
    -wallet C:\app\oracle\product\wallets\my_wallet
    -password "MiPasswordSecreto123"
```



```
-trusted_cert
-cert C:\Ruta\Hacia\certificado_raiz.cer
```

4. Otorgar accesos en la Base de Datos para la ruta Windows Conéctate a SQL*Plus o SQL Developer como SYSDBA y ejecuta el siguiente bloque para dar acceso al esquema a la ruta de Windows (nota que se usa el prefijo file: seguido de la ruta con barras invertidas \ o normales /, pero Oracle en Windows acepta barras normales para estandarizar):

```
BEGIN
  --
  DBMS_NETWORK_ACL_ADMIN.APPEND_WALLET_ACE(
    wallet_path    => 'file:C:\app\oracle\product\wallets\my_wallet',
    ace            => xs$ace_type(
                     privilege_list =>
xs$name_list('use_client_certificates'),
                     principal_name => 'APP_USER',           -- Tu
usuario de BD
                     principal_type => xs_acl.p_type_db
                     )
  );
  --
END;
```

5. Modificación en el Código PL/SQL (UTL_HTTP) En tu procedimiento almacenado, la llamada a UTL_HTTP.SET_WALLET debe apuntar textualmente a la ruta de Windows establecida:

```
BEGIN
  --
  -- Apuntar al Oracle Wallet en el sistema de archivos de Windows
  UTL_HTTP.SET_WALLET('file:C:\app\oracle\product\wallets\my_wallet', NULL);
  --
  -- Continuar con la configuración del Proxy y Request habitual
  UTL_HTTP.SET_PROXY('://miempresa.com', NULL);
  v_http_req := UTL_HTTP.BEGIN_REQUEST(v_url, 'POST', 'HTTP/1.1');
  -- ...
```

Consejo de Seguridad para Windows

Asegúrate de que la carpeta **C:\app\oracle\product\wallets\my_wallet** tenga los permisos de seguridad de Windows correctos.

El usuario de Windows que ejecuta el servicio de la base de datos (generalmente un usuario dedicado de Oracle, SYSTEM o Servicio de red) debe tener permisos de Lectura y Escritura sobre esa carpeta y sus archivos (cwallet.sso y ewallet.p12).

CERTIFICADOS .CER

Para configurar las peticiones HTTPS en Oracle utilizando el paquete UTL_HTTP y un Oracle Wallet, el motor de la base de datos no necesita el certificado del sitio final en sí, sino los Certificados Raíz (Root CA) e Intermedios de la entidad emisora que firmó esa web. Si no los incluyes, Oracle rechazará la conexión con el error clásico ORA-29024: Certificate validation failure. [rostantechnologies](#), [bvstools](#)

A continuación, se explica el método más fácil y directo para obtenerlos desde tu navegador de internet, o bien cómo generarlos si el endpoint es un servidor propio.

Método 1

Obtenerlos desde Google Chrome (Sitios de Internet o APIs Externas).

Si te vas a conectar a un Webhook o API pública (por ejemplo: <https://clienteexterno.com>), puedes descargar la cadena de confianza siguiendo estos pasos: [smppcenter](#), [appdome](#)

1. Entra a la URL: Abre Google Chrome y navega a la dirección del API (asegúrate de incluir el protocolo <https://>). [appdome](#), [smppcenter](#)
2. Abre el Visor del Certificado:
 - En la barra de direcciones, haz clic en el icono del candado o los controles deslizantes a la izquierda de la URL.
 - Selecciona "La conexión es segura" y luego haz clic en "El certificado es válido". [smppcenter](#), [appdome](#)
3. Identifica la Cadena de Confianza:
 - En la ventana emergente que se abre (Visor de Certificados), ve a la pestaña Detalles. Verás una sección jerárquica llamada Jerarquía de certificados que suele tener 3 niveles:
 - Nivel superior: El certificado Raíz (Root) — Éste es el más importante para Oracle.
 - Nivel medio: El certificado Intermedio.
 - Nivel inferior: El certificado del sitio final (Server). [bvstools](#), [smppcenter](#), [appdome](#)
4. Exportar el Certificado Raíz:
 - Selecciona el componente del nivel superior (el Raíz).
 - Haz clic en el botón Exportar... (abajo a la derecha).
 - En la ventana de guardado de Windows, asígnale un nombre descriptivo (ej: [raiz_api.cer](#)). Muy Importante: En el tipo de archivo, asegúrate de seleccionar "Binario codificado DER (.cer)" o "X.509 codificado en Base64 (.cer)". Oracle acepta ambos formatos para importarlos mediante `orapki`. [stackoverflow](#), [smppcenter](#), [appdome](#), [stackoverflow](#)
5. Exportar el Certificado Intermedio (Opcional pero Recomendado): Haz clic sobre el nivel del medio en la jerarquía y repite el proceso de exportación guardándolo como [intermedio_api.cer](#). [smppcenter](#). Mueve estos archivos .cer obtenidos a tu servidor de base de datos Windows o Linux para procesarlos con los comandos `orapki wallet add` que vimos en las respuestas previas. [rostantechnologies](#)

Método 2

Obtenerlos usando la Línea de Comandos (Cualquier Sistema Operativo).

Si prefieres la automatización o no tienes una interfaz gráfica, puedes extraer el certificado raíz directamente desde una consola de comandos (CMD en Windows o Bash en Linux) utilizando la herramienta universal OpenSSL:

```
# Reemplaza ://clienteexterno.com por el dominio real de tu API
openssl s_client -showcerts -connect ://clienteexterno.com:443 </dev/null
```

Este comando imprimirá bloques de texto que inician con -----BEGIN CERTIFICATE----- y terminan con -----END CERTIFICATE----- . Puedes copiar el bloque correspondiente al certificado raíz emisor, pegarlo en un bloc de notas y guardarlo con la extensión .cer o .pem. [rostantechnologies](#)

Método 3:

¿Qué pasa si el API es interna? (Crear tus propios certificados)

Si la base de datos debe conectarse a un servidor web HTTPS propio de tu empresa que no usa una entidad pública certificada (como DigiCert o Let's Encrypt), significa que tu empresa tiene una Entidad Certificadora Interna (CA Privada) o el sitio usa un Certificado Auto-firmado.

- Si es una CA Interna: Debes ponerte en contacto con el equipo de Infraestructura, Seguridad o Redes de tu empresa y solicitarles: "El certificado público de la entidad raíz local en formato .cer X.509". Ellos te darán el archivo oficial directamente.
- Si es Auto-firmado: El certificado del servidor web del API actúa como su propia raíz. Por lo tanto, usando el **Método 1** (Chrome), exporta directamente el único nivel disponible de la jerarquía en formato .cer e impórtalo como certificado de confianza (-trusted_cert) en tu Oracle Wallet.
[rostantechnologies](#)